Revisão AP1

Java

HERANÇA

↳ método de uma sub-classe aproveitar propriedades de uma super classe

Encapsulamento (modificadores de visibilidade)

Public - Todos

Private: Apenas a classe

Package - Pacote

Protected - Todas subclasses que herdaram independente do pacote

→ restrição de Propriedades (NIVEIS DE ACESSO)

Polimorfismo (1 metodo Diferentes Formas)

Sobre escrita

- OOverride
- EXECUÇÃO
- mesma assinatura e nome, mas retorna resultados Diferentes

Sobrecarga

- Somar (int x, int y)
- Somar (int x) { x + x + 1 }
- COMPILAÇÃO
- métodos com mesmo nome mas assinaturas (parâmetros) distintos

Java Conventions

- Espaços (int x, int y) {

}

. CamelCase : classe → PessoaJuridica

método → calcularExtratoCliente

- comentários

CheckSTAYLE

- Plugin / Ferramentas em ide que garante o Java code conventions.

Clean Code

. Boas Práticas de codar

- Varia Veis claras e explicativas

Exceptions

(quanto menos if, mais OO! o código)

- THROW → único, pelo código

- TRY { ...
  } CATCH { (Exception e) }

  System (e.message);

SOLID

→ código +: legível, fácil de manter, reutilizável, escalável

- Single Responsability (Separar Responsabilidade)

  - 1 classe apenas 1 Responsabilidade

- OPEN/CLOSED (add comportamento, mas não muda o código)

  - ABERTOS para extensão / fechados para alteração

- LISKOV Substitution (classe filha não altera as coisas da PAI)

. Se a SuperClass faz X a subclass deve fazer X

- Interface Segregation

. Interfaces pequenas e específicas

- Dependency Inversion

. Não instanciar uma classe em outra

Resumo de Solid

- Cada classe tem um Papel claro
- Facil manter e atualizar (Sem comprometer)
- Facil de testar